

PRISM — Development Phases

Precise Review & Intelligent Scoring Module

IBM Bob 2.0 Hackathon — AI Code Review Coach

Overview

PRISM is built across 9 sequential phases. Each phase is runnable and demonstrable before the next begins — no phase requires the next one to exist. The order is driven entirely by dependency: environment before pipeline, pipeline before RAG, RAG before datasets, everything before the UI.

Quick Reference — Phase Order

Phase	Name	One-Line Summary
Phase 1	Environment & Infra	Install Ollama, wire GitHub MCP, fill .env, confirm health check passes
Phase 2	Core Pipeline (no RAG)	Diff ingestion, hunk chunker, Granite prompt, JSON findings, dedup & severity rank
Phase 3	MCP Server Shell	Register all 6 tools in server.py; Bob can call prism_analyze_pr and get real output
Phase 4	RAG Layer	ChromaDB + Ollama embedder + tree-sitter chunker + cross-encoder reranker
Phase 5	Cold-Start Datasets	Download CodeReviewer sample; write OWASP + smells JSONL; build cold-start index
Phase 6	FastAPI Backend	REST API wrapping MCP tools; session storage; CORS ready for frontend
Phase 7	Dashboard (Frontend)	Next.js: PR input form, risk gauge, diff viewer with inline annotations, findings table
Phase 8	GitHub Comment Posting	Map findings to diff positions; batch create_pull_request_review; dry-run gate
Phase 9	Polish & Demo Prep	Staged demo repo, citation rendering, PR summary prompt, judge demo script rehearsal

Phase 1 — Environment & Infrastructure

Goal

Get every external dependency installed, authenticated, and verified before a single line of application code is written. This phase ends when prism_health_check returns all green.

Why first

Nothing in any later phase can be tested without Ollama running and a valid GitHub token. Discovering a broken dependency in Phase 4 wastes hours. Discovering it here costs 10 minutes.

What gets built

Install Ollama and pull both required models: granite3-dense:8b (generation) and nomic-embed-text (embeddings). Verify Ollama is reachable at <http://localhost:11434>.

Create a GitHub Personal Access Token (classic PAT) with repo and write:discussion scopes. This is free on any GitHub account.

Install the GitHub MCP server via npx (@modelcontextprotocol/server-github) and register it in .bob/mcp.json. Verify Bob can call get_pull_request on a test PR.

Create a HuggingFace account and generate a free read token (hf_...). This is the fallback embedder if Ollama is unreachable. Free tier, no credit card.

Copy .env.example to .env and fill in GITHUB_TOKEN and optionally HF_TOKEN. All other variables have working defaults.

Create the Python virtual environment, install requirements.txt (mcp, chromadb, sentence-transformers, tree-sitter, tiktoken, fastapi, datasets).

Write and run prism_health_check — a single script that pings Ollama, instantiates a ChromaDB client, and makes one GitHub API call. All three must return OK.

Exit criteria

prism_health_check returns: Ollama OK, ChromaDB OK, GitHub token OK. You can run ollama generate and get a response. The GitHub MCP server can fetch a real PR diff.

Phase 2 — Core Pipeline (No RAG)

Goal

Build the end-to-end review loop — diff in, findings out — using a plain prompt to Granite, no retrieval. This is the minimal viable PRISM: it works on any PR from the moment this phase ends.

Why second

The pipeline is the backbone. RAG, datasets, and the UI are all improvements on top of it. If anything in a later phase breaks, falling back to this bare pipeline still gives you a demonstrable product.

What gets built

diff_parser.py: Parse a raw unified diff string into a list of HunkChunk objects. Each chunk carries: file_path, language (detected from extension), hunk_header, the diff lines, and the surrounding 30 lines of full-file context.

token_counter.py: tiktoken-based utility to count tokens in any string and split oversized hunks into overlapping 800-token sub-hunks with 100-token overlap.

Prompt builder: Assembles the PRISM system prompt + diff hunk into a request for Granite. No context injection yet — just the diff and the instruction to output a JSON array of findings.

Ollama caller: HTTP POST to localhost:11434/api/generate. Parses the streamed response, extracts the JSON array. One retry with a stricter JSON-only instruction if parsing fails on the first attempt.

Finding schema: Each finding has severity (Blocker/Major/Minor/Nit), category (security/logic/maintainability/style/tests), line_start, line_end, title, explanation, suggestion, citation (null at this stage).

dedup.py: Merge findings across all hunks. Two findings are duplicates if they share the same file + category and their line_start values are within 5 lines. Keep the higher-severity one.

Risk score: Blockers x10 + Majors x3 + Minors x1 + Nits x0.2, capped at 100.

Smoke test script (scripts/test_review.py): Takes a PR URL, runs the full Phase 2 pipeline, prints findings to stdout. This is the validation checkpoint for this phase.

Exit criteria

python scripts/test_review.py --pr <URL> prints a JSON list of findings with correct severity tags. At least one Blocker or Major finding on a PR that has a known security issue. Risk score is non-zero and sensible.

Phase 3 — MCP Server Shell

Goal

Wrap the Phase 2 pipeline inside the official MCP server so Bob can call it as a tool. From this point on, you interact with PRISM via Bob tool calls, not Python scripts directly.

Why third

The MCP server is the integration point between Bob and everything PRISM does. Wrapping the pipeline here early means every subsequent phase (RAG, datasets, backend) is immediately accessible to Bob without additional wiring.

What gets built

prism_mcp/server.py: The entrypoint. Imports the mcp SDK and registers all 6 tools using the @server.tool() decorator. Started by Bob as a stdio subprocess via .bob/mcp.json.

prism_health_check tool: Pings Ollama, ChromaDB, and GitHub token. Returns a structured JSON status object. Bob calls this automatically on workspace open.

prism_analyze_pr tool: Full input/output schema defined. At this stage it calls the Phase 2 pipeline directly. Returns the structured review result with findings, risk score, and summary stub.

prism_build_corpus tool: Stub at this stage — returns {status: not_implemented}. Will be filled in Phase 4.

prism_retrieve_context tool: Stub at this stage — returns empty context_chunks array. Phase 4 fills this in.

prism_generate_review tool: Thin wrapper around the Phase 2 Ollama caller. Takes diff_hunk + context_chunks (ignored for now) and returns findings JSON.

prism_post_review tool: Stub at this stage. Will call GitHub MCP in Phase 8.

.bob/mcp.json registration: Adds the prism server entry (python -m prism_mcp.server) and the GitHub MCP server entry (npx @modelcontextprotocol/server-github).

Exit criteria

Bob can call prism_analyze_pr with a PR URL and receive a structured findings JSON. prism_health_check returns all green. All 6 tools are registered and return valid (even if stubbed) responses.

Phase 4 — RAG Layer

Goal

Add retrieval-augmented generation to the review pipeline. Every finding is now grounded in real repo context — style docs, past review comments, and similar code patterns — not just the LLM's training data.

Why fourth

RAG is what turns PRISM from a linter wrapper into a quality coach. But it needs the pipeline (Phase 2) and the MCP server (Phase 3) to exist first. The retrieval machinery can be tested with just the repo's own CONTRIBUTING.md — no datasets needed yet.

What gets built

chunker.py: tree-sitter based AST chunker. Splits source files at function/method/class boundaries. Falls back to heading-boundary splits for Markdown/config. Attaches metadata: file_path, start_line, end_line, language, symbol_name, chunk_type.

embedder.py: Calls Ollama's /api/embeddings endpoint with model nomic-embed-text. Batches 32 chunks per request. Falls back to HuggingFace Inference API (all-MiniLM-L6-v2) if Ollama is unreachable.

vector_store.py: ChromaDB wrapper. Manages 6 collection types (style, history, source per repo; cold-start reviews, OWASP, smells globally). Upserts by content SHA256 hash for idempotent re-indexing.

reranker.py: Loads cross-encoder/ms-marco-MiniLM-L-6-v2 locally via sentence-transformers. Scores all (query, chunk) pairs and returns top-k sorted by reranker score. Guaranteed 3 slots for repo-specific chunks.

prism_build_corpus (now implemented): Fetches CONTRIBUTING.md, style guide, and linter configs from the repo via GitHub MCP. Fetches the last 50 merged PRs and their review comment threads. Chunks, embeds, and upserts everything into ChromaDB. Idempotent — safe to re-run.

prism_retrieve_context (now implemented): Given a diff hunk, queries ChromaDB collections, runs the reranker, and returns the top-6 context chunks with source labels. These are injected into the review prompt by prism_generate_review.

Prompt update: The review prompt template now includes the retrieved context block (capped at 1500 tokens) between the system instruction and the diff hunk. citation field in findings is now populated from source_label of the top matching chunk.

Exit criteria

`prism_build_corpus` on a repo with a `CONTRIBUTING.md` succeeds and populates at least 10 style chunks. `prism_retrieve_context` returns 6 chunks for a sample diff hunk. At least one finding in a review carries a citation field pointing to `CONTRIBUTING.md` or a past PR.

Phase 5 — Cold-Start Datasets

Goal

Build the cold-start vector store that gives PRISM meaningful grounding for any repo, even one with zero review history. This runs once as a setup step.

Why fifth

The retrieval machinery (Phase 4) must exist before datasets can be indexed into it. Datasets improve retrieval quality but are not required to test retrieval. Putting this here keeps the critical path short and defers the 15-minute embedding job to after the pipeline is validated.

What gets built

`scripts/download_datasets.py`: Uses HuggingFace datasets library to download the Microsoft CodeReviewer dataset (MIT license, free, ~150k rows). Samples 50,000 rows evenly across Python, Java, JavaScript, TypeScript, and Go. Writes to `data/cold_start/codereviewer_sample.jsonl`.

`data/cold_start/owasp_rules.jsonl`: Hand-written file covering 8 OWASP Top 10 rules (A01-A10 subset). Each entry has `rule_id`, `rule_name`, `language`, `pattern_description`, `example_bad`, `why_bad`, `fix`, `source_label`. Covers Python, Java, JavaScript, TypeScript patterns.

`data/cold_start/code_smells.jsonl`: Hand-written file covering ~40 code smells from the catalog (Long Method, Dead Code, Duplicate Code, Feature Envy, etc.). Each entry has `smell_id`, `smell_name`, `category`, `detection_signal`, `why_bad`, `fix`, `source_label`.

`scripts/setup_cold_start.py`: Reads all three JSONL files, chunks them, embeds them via the Phase 4 embedder, and upserts into three ChromaDB collections: `cold_start_reviews`, `cold_start_owasp`, `cold_start_smells`. Expected runtime: ~15 minutes on a modern laptop.

Corpus size after Phase 5: ~50,080 chunks total, ~180 MB on disk. `cold_start_reviews` (~50k chunks) provides real (diff, review comment) pairs as precedent. `cold_start_owasp` (40 chunks) gives security rule grounding. `cold_start_smells` (40 chunks) gives maintainability grounding.

Exit criteria

`python scripts/setup_cold_start.py` completes without error. `prism_corpus_status` shows `cold_start_loaded`: true with chunk counts matching expectations. A review on a brand-new repo with no history returns at least one finding with an OWASP or code-smell citation.

Phase 6 — FastAPI Backend

Goal

Expose PRISM's MCP tools as a REST API so the Next.js frontend can call them over HTTP. The backend is a thin translation layer — it does not implement any review logic itself.

Why sixth

The backend only makes sense once there is a complete, tested pipeline (Phases 2-5) to expose. Building it before the pipeline is stable leads to constant API contract churn.

What gets built

backend/main.py: FastAPI app. Configures CORS (allowing localhost:3000). Mounts three routers: /review, /corpus, /health.

POST /review/analyze: Accepts {pr_url, post_comments, language_hint}. Calls prism_analyze_pr via the MCP client. Stores result in an in-memory session dict keyed by a UUID. Returns {review_id, status: processing} immediately (async).

GET /review/{review_id}: Returns the full review result once complete, or {status: processing} if still running. Frontend polls this every 2 seconds.

POST /review/{review_id}/post: Calls prism_post_review with the stored findings. dry_run defaults to true; requires explicit {dry_run: false} to actually post.

POST /corpus/build: Accepts {owner, repo, force_rebuild}. Calls prism_build_corpus. Returns corpus statistics.

GET /corpus/status: Calls prism_corpus_status. Returns indexed repos and cold-start load state.

GET /health: Calls prism_health_check. Returns dependency status for the dashboard health panel.

backend/models/schemas.py: Pydantic models for all request/response bodies. Shared with the MCP tool schemas to keep them in sync.

Exit criteria

uvicorn backend.main:app --reload starts cleanly. curl POST /review/analyze with a valid PR URL returns a review_id. Polling GET /review/{id} eventually returns a full findings payload. GET /health returns all-green.

Phase 7 — Dashboard (Frontend)

Goal

Build the judge-facing UI. The dashboard is the primary demo surface — everything the judge sees lives here. It must be clean, fast, and scannable in under 10 seconds.

Why seventh

The frontend calls the backend API. It cannot be built meaningfully until the API (Phase 6) is delivering real data. Building UI against mock data earlier is possible but wastes time when the real data shape changes.

What gets built

frontend/app/page.tsx: Landing page. Single text input for the GitHub PR URL. Submit button calls POST /review/analyze. Shows a loading spinner and per-hunk progress feedback while the review runs.

frontend/app/review/[id]/page.tsx: The main results page. Polls GET /review/{id} until complete. Renders all panels below.

PR Header component: Shows PR title, author, base-to-head branch, link back to GitHub.

RiskScore component: Circular gauge 0-100. Green below 20, yellow 20-50, red above 50. The number is the first thing judges see.

FindingsPanel component: Two charts side by side — a bar chart of findings by severity (Blocker/Major/Minor/Nit) and a pie chart by category (security/logic/maintainability/style/tests). Uses recharts (free, MIT).

DiffViewer component: Uses react-diff-viewer-continued (MIT). Shows the full unified diff with syntax highlighting. Lines with findings have a colored left-border indicator. Clicking an indicator expands an inline card showing: severity badge, title, explanation, suggestion, and citation label.

FindingsList component: Sortable, filterable table of all findings. Columns: severity, file, line, category, title. Clicking a row scrolls the diff viewer to that line.

CorpusStatus component: Small panel in the footer. Shows which collections are loaded, chunk counts, last-indexed timestamps. Serves as a live health indicator during the demo.

Post to GitHub button: Calls POST /review/{id}/post with dry_run: false. Disabled until the review is complete. Shows a confirmation dialog before posting. Displays a link to the GitHub PR after posting.

Exit criteria

npm run dev starts at localhost:3000. Pasting a PR URL and submitting shows the full results page with risk score, charts, and diff viewer with at least one inline finding annotation. The corpus status panel shows real data.

Phase 8 — GitHub Comment Posting

Goal

Post PRISM's findings as a formal GitHub PR review — line-anchored inline comments, not just a summary comment. This is what makes PRISM a real workflow integration rather than a side dashboard.

Why eighth

Posting to GitHub is a destructive operation — it creates real comments on real PRs. It must only be wired up after finding quality is confirmed good (Phases 2-7). The dry_run default throughout all earlier phases is intentional.

What gets built

Diff position mapper: GitHub's create_pull_request_review requires a position integer (line offset from the start of the diff hunk, not an absolute file line number). This mapper builds a

lookup table from the diff parser output and maps each finding's `line_start` to the correct GitHub position value.

Comment formatter: Renders each finding as a formatted GitHub comment body. Format: bold severity + title on line 1, blank line, explanation paragraph, blank line, suggested fix in a fenced code block if present, italic citation label at the bottom.

`prism_post_review` (now implemented): Calls GitHub MCP `create_pull_request_review` with event type `COMMENT` (non-blocking). The review body is the PR summary paragraph. Each finding becomes one inline comment at the correct diff position. All comments are batched into the single review call.

Demo repo safety gate: PRISM checks that the target repo matches `DEMO_REPO_OWNER/DEMO_REPO_NAME` from `.env` before allowing `dry_run: false`. This prevents accidentally posting on a real team repo during the demo.

Incremental learning hook: After a successful post, schedule a corpus update for when the PR is merged. This is a background task that appends the posted comments to the repo's history collection — closing the feedback loop.

Exit criteria

On the demo repo: `prism_post_review` with `dry_run: false` creates a real GitHub PR review. The review appears in the PR timeline with inline comments anchored to the correct lines. The comment body includes a citation label. At least one comment is attached to a non-trivial diff line.

Phase 9 — Polish & Demo Prep

Goal

Make the end-to-end flow judge-ready. This is not about adding features — it is about tightening what exists, removing rough edges, and staging a controlled demo environment that works reliably under pressure.

Why last

Polish cannot happen until the product is complete. Demo prep requires a working product to prepare. This phase is always last because its inputs are all the other phases.

What gets built

Staged demo repo (`prism-demo-repo` on GitHub): Pre-populate with 5-10 merged PRs that each demonstrate a specific finding category. One PR has a SQL injection bug (Blocker, security). One has a missing null check (Major, logic). One violates `CONTRIBUTING.md` rule (Minor, style). One has dead code (Nit, maintainability). One is missing test coverage (Major, tests). Keep one open PR ready for the live demo.

Citation rendering: Verify that at least 3 of the 5 staged PRs produce findings with non-null citation fields. The demo's key moment is showing 'per `CONTRIBUTING.md` section 3.2' or 'similar issue in PR #7' — make sure this actually fires for the demo PRs.

PR summary prompt: Add the dedicated summary generation call (one prompt per PR, not per hunk) that produces the one-paragraph overview and bullet list of risks. Wire the output into the dashboard's summary panel and the GitHub review body.

Error state UI: Add graceful error handling in the dashboard for the three most likely failures: Ollama unreachable (show install instructions), GitHub token expired (show how to refresh), PR too large (show which files were skipped).

Performance check: Run the full pipeline on the live demo PR and time it. Target: under 60 seconds for a PR with 5-10 changed files. If over 90 seconds, profile and identify the bottleneck (usually embedding batch size or Ollama model load time).

Judge demo script rehearsal: Run through the exact 5-step demo sequence twice. Step 1: paste PR URL, click Analyze. Step 2: watch live progress. Step 3: review dashboard, click a citation annotation. Step 4: show the diff viewer, point out the inline finding on the SQL injection line. Step 5: click Post to GitHub, open GitHub and show the review comment appearing in real time.

Backup plan: If Ollama is slow on the demo machine, pre-generate and cache the review result for the demo PR. The dashboard can load from cache; the live GitHub posting still runs live.

Exit criteria

End-to-end demo runs cleanly twice in a row, under 60 seconds each. The live PR produces at least one finding with a citation. The GitHub posting step works on the demo repo. The dashboard looks clean with no console errors or broken states.